GPU 利用率与性能手册

GPU Utilization / Performance Handbook

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Memory Hierarchy	内存层次结构	
Tensor Cores	张量核心	
Throughput	吞吐量	
Warps	线程束	
Shared Memory	共享内存	
Performance	性能	
Occupancy	占用率	
Registers	寄存器	
Latency	延迟	
Mental Model	理解框架	
Tensor Core	张量核心	
Coalescing	合并访问	
Profiling	性能剖析	
Register	寄存器	
Tiling	分块	
Kernel	内核	
Resident	驻留	
Synchronization	同步	
Warp	线程束	
Memory Bandwidth	内存带宽	
GPU Utilization	GPU 利用率	
FLOP	浮点运算	
Generation	生成	
Matrix	矩阵	
SM	流式多处理器	SM
Streaming Multiprocessor	流式多处理器	SM
Register Pressure	寄存器压力	

英文原词	中文译名	缩写／符号
Eligible Warps	可调度线程束	
Kernel Launch	内核启动	
Device Memory	设备内存	
Local Memory	局部内存	
Active Lanes	活动通道	
Precision	精度	
Residency	驻留状态	
Spilling	溢出	
SIMT	单指令多线程	SIMT
GEMM	通用矩阵乘法	GEMM
MMA	矩阵乘加	MMA
Compute-bound	计算受限	
Memory-bound	内存受限	
Data Movement	数据移动	
Transformer	Transformer 模型	Transformer
Attention	注意力机制	Attention
Tensor	张量	
Model	模型	
Memory Traffic	内存流量	
FLOP Count	浮点运算次数	
Matrix Multiplication	矩阵乘法	
Memory Access	内存访问	
Embedding	嵌入	
Serving	推理服务	
Decode	解码	
Thread Block	线程块	
Register File	寄存器文件	
CUDA Core	CUDA 核心	
Vector	向量	

英文原词	中文译名	缩写／符号
SIMD	单指令多数据	SIMD
Dispatch	分派	
Pipeline	流水线	
Values	值	
L2 Cache	二级缓存	
Working Set	工作集	
Global Memory	全局内存	
Memory Footprint	内存占用	
Dense Attention	稠密注意力	
FlashAttention	FlashAttention（方法名保留）	
GPU Generations	GPU 代际	
Floating-point	浮点	
Core	核心	
Launch Overhead	启动开销	
GPU Generation	GPU 代际	
Accumulation	累加	
Inference	推理	
Training	训练	
Generational	代际的	
FP32	32 位浮点	FP32
Scaling	缩放	
Wall-clock Time	实际耗时	
Bandwidth-bound	带宽受限	
Models	模型	
Benchmark	基准测试	
Tokens	词元	
Dtype	数据类型	
Arithmetic Intensity	算术强度	
Operational Intensity	运算强度	

英文原词	中文译名	缩写／符号
Batch Size	批大小	
Roofline Model	屋顶线模型	
Normalization	归一化	
Softmax	Softmax 归一化	Softmax
Batching	批处理	
Prefill	预填充	
Query	查询	Q
Wavefront	波前	
CUDA Cores	CUDA 核心	
Saturation	饱和	
Evidence	证据	
Exact Attention	精确注意力	
Profiler	性能剖析器	

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

GPU 利用率与性能手册 · 第 8 页

如何阅读本手册 · 第 9 页

内容 · 第 10 页

内容续 · 第 11 页

1 GPU 架构对 AI 的实际意义 · 第 12 页

2 CPU和GPU优化不同目标 · 第 13 页

3 从内核启动（Kernel Launch）到硬件工作 · 第 14 页

4 流式多处理器（Streaming Multiprocessor）是执行中心 · 第 15 页

5 线程束（Warps）和 SIMT · 第 16 页

6 分流车道和活动车道 · 第 17 页

7 Warp 调度程序和符合条件的 Warp · 第 18 页

8 延迟（Latency）隐藏并不是延迟减少 · 第 19 页

9 驻留、执行和进展 · 第 20 页

10 占用：确切的定义 · 第 21 页

11 为什么 100% 入住率不是目标 · 第 22 页

12 个寄存器（Registers）：快速、线程本地状态 · 第 23 页

13 寄存器压力（Register Pressure）、溢出（Spilling）和本地内存 · 第 24 页

14 共享内存（Shared Memory）：软件管理的片上存储 · 第 25 页

15 Cache：L1和L2不是共享内存（Shared Memory） · 第 26 页

16 HBM 和设备内存（Device Memory）：容量不是带宽 · 第 27 页

17 内存层次结构（Memory Hierarchy）是一个数据移动问题 · 第 28 页

18 合并：线程束（Warp）如何请求内存 · 第 29 页

19 执行管道：指令不使用一种资源 · 第 30 页

20 个 Tensor 核心：矩阵（Matrix） MMA，而不是神奇的 AI 核心 · 第 31 页

21 Precision改变可用的执行路径 · 第 32 页

22 GEMM 如何映射到 GPU 层次结构 · 第 33 页

23 指令级和线程级并行 · 第 34 页

24 同步（Synchronization）会暴露空闲时间 · 第 35 页

25 内核（Kernel）启动和异步执行 · 第 36 页

26 “GPU 利用率（GPU Utilization）”的实际含义是什么 · 第 37 页

27 占用率（Occupancy）、活动量和吞吐量（Throughput）不同 · 第 38 页

28 峰值 TFLOP 与实现的吞吐量（Throughput） · 第 39 页

29 计算限制和内存限制是工作负载属性 · 第 40 页

30 个 AI 工作负载对 GPU 的不同部分产生压力 · 第 41 页

31 NVIDIA 与 AMD：名称不同，原则不变 · 第 42 页
32 代架构改变了机制 · 第 43 页
33 有效执行示例：61,440 个逻辑线程 · 第 44 页
34 工作占用示例：资源限制相互作用 · 第 45 页
35 种常见误解 · 第 46 页
36 实用的理解框架（Mental Model） · 第 47 页
参考文献和进一步阅读 · 第 48 页
参考图：针对哪个问题阅读什么内容 · 第 50 页

GPU 利用率与性能手册

对应原稿第 1 页

了解 GPU AI 架构

从线程束（Warps）和张量核心（Tensor Cores）到内存层次结构（Memory Hierarchy）和吞吐量（Throughput）

关于 AI 工作负载如何映射到真实 GPU 硬件的技术手册：流式多处理器、线程束（Warps）、调度程序、执行管道、张量核心（Tensor Cores）、寄存器（Registers）、共享内存（Shared Memory）、缓存、HBM、占用、延迟（Latency）隐藏和性能（Performance）指标。

如何阅读本手册

对应原稿第 2 页

当四个不同的问题分开时，GPU 架构最容易推理：

1. 有什么工作？程序和内核（Kernel）定义线程、块、指令、依赖关系和内存访问。

2.常驻什么工作？寄存器（Registers）、共享内存（Shared Memory）、架构限制和启动配置决定了执行引擎上可以同时存在多少个块和线程束（Warps）。

3.准备发行什么作品？常驻线程束（Warp）仍然可能因数据、依赖性、同步（Synchronization）或其他约束而停滞。调度员在符合条件的工作中进行选择。[1,3]

4. 哪些硬件资源限制了进度？算术管道、Tensor Core、内存带宽（Memory Bandwidth）、延迟（Latency）、指令发布、同步（Synchronization）或主机端开销可能成为不同工作负载下的瓶颈。[2,3]

核心教训是 GPU 是一台吞吐量（Throughput）机器，而不是一个巨大的算术单元。高峰值 FLOP 率、高占用率（Occupancy）或显示“100% GPU 利用率（GPU Utilization）”的仪表板本身并不能证明 AI 工作负载正在高效运行。这些量衡量的是不同的事物。[2,4]

> 范围。本手册建立了理解人工智能系统系列其余部分所需的执行理解框架（Mental Model）。内存层次结构（Memory Hierarchy）、Roofline 分析、合并、共享内存平铺、寄存器（Register）/占用调整、Tensor Core 编程和分析稍后都会收到更深入的独立手册。这里对它们的介绍仅足以使 GPU 架构保持一致。

供应商大会。 NVIDIA CUDA 术语用于具体执行路径，因为它在人工智能系统中广泛遇到。供应商特定的事实被标记。 AMD 术语用作交叉检查，以便一般 GPU 原则不会意外地呈现为 CUDA 法则。[8,9]

符号。在 NVIDIA 硬件上，一个 warp 包含 32 个线程。对于具有 T线程的块，

$$\left\lceil \frac{T}{32} \right\rceil .$$

$N_{\text{warps}}=$

32

SM 的确切资源限制、执行管道的数量和类型、缓存组织和矩阵（Matrix）指令因架构生成（Generation）而异。[1]

内容

对应原稿第 3 页

1 GPU 架构对 AI 的实际意义 5 2 CPU 和 GPU 优化不同的目标 6 3 从内核启动（Kernel Launch）到硬件工作 7 4 流式多处理器（Streaming Multiprocessor）是执行中心 8 5 Warps 和 SIMT 9 6 分歧和活动通道（Active Lanes） 10 7 Warp 调度程序和合格的 warps 11 8 延迟（Latency）隐藏不是减少延迟 12 9 驻留、执行和进度 13 10 占用：确切定义 14 11 为什么 100% 占用不是目标 15 12 寄存器（Registers）：快速、线程本地状态 16 13 寄存器压力（Register Pressure）、溢出（Spilling）和本地内存 17 14 共享内存（Shared Memory）：软件管理的片上存储 18 15 缓存：L1 和 L2 不是共享内存19 16 HBM 和设备内存（Device Memory）：容量不是带宽 20 17 内存层次结构（Memory Hierarchy）是一个数据移动问题 21 18 合并：线程束（Warps）如何请求内存 22 19 执行管道：指令不使用一种资源 23 20 张量核心（Tensor Cores）：矩阵（Matrix） MMA，而不是神奇的 AI 核心 24 21 精度（Precision）会改变可用的执行路径 25 22 如何 GEMM 映射到 GPU 层次结构 26 23 指令级和线程级并行性 27 24 同步（Synchronization）可能会暴露空闲时间 28

内容续

对应原稿第 4 页

25 内核（Kernel）启动和异步执行 29 26 “GPU 利用率（GPU Utilization）”实际上意味着什么 30 27 占用率（Occupancy）、活动和吞吐量（Throughput）不同 31 28 峰值 TFLOP 与实现的吞吐量 32 29 计算密集型和内存密集型是工作负载属性 33 30 AI 工作负载对 GPU 的不同部分施加压力 34 31 NVIDIA 与AMD：名称不同，原则不变 35 32 架构代际改变了机制 36 33 有效执行示例：61,440 个逻辑线程 37 34 有效占用示例：资源限制相互作用 38 35 常见误解 39 36 实用理解框架（Mental Model） 40 参考资料和进一步阅读 41 参考图：哪个来源回答哪个问题 42

手册的前半部分解释了机器。后半部分解释了为什么常见的性能（Performance）数据可能不一致。这种分离是经过深思熟虑的：架构描述了存在的资源；性能分析询问哪些资源实际上限制了特定的工作负载。

1 GPU 架构对 AI 的实际意义

对应原稿第 5 页

人工智能模型（Model）被表示为张量（Tensor）运算，但硬件不会将“Transformer 层”或“注意力机制（Attention）块”作为原语执行。框架、编译器和库最终将工作转化为设备操作和内核。这些内核向 GPU 执行系统公开并行线程、指令、内存访问和同步（Synchronization）。[1]



内核（Kernel）

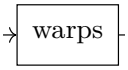


网格

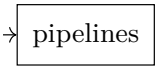


块





线程束（Warps）



管道

抽象是分层的，因为硬件是分层的。块提供了可以放置在 SM 上的单元。 Warps 提供 SIMT 执行模型（Model）使用的本机线程分组。寄存器（Registers）和共享内存（Shared Memory）提供片上状态。缓存和设备内存（Device Memory）提供数据。不同的流水线执行不同的指令类。[1]

这就是为什么具有相同 FLOP 计数的两个内核可能具有截然不同的运行时间。它们可能会生成不同的内存流量（Memory Traffic），暴露不同数量的独立工作，使用不同的执行单元，或者遭受不同数量的同步（Synchronization）和发散。 GPU 的峰值能力描述了天花板。工作负载到架构的映射决定了有多少有用的工作接近这些上限。[2,3]

> 理解框架（Mental Model）。 GPU 架构是将并行工作和数据移动（Data Movement）转化为吞吐量（Throughput）的机制。人工智能性能（Performance）是工作负载-硬件映射的属性，而不是单个规格编号的属性。

2 CPU和GPU优化不同目标

对应原稿第 6 页

CPU 和 GPU 设计强调不同类型的进步。 CPU 投入了大量资金来使相对较小的指令流集快速前进。 GPU 接受较低的每线程性能（Performance），因此它可以保持更多的线程处于运行状态并提供高聚合吞吐量（Throughput）。[1,2]

CPU强调：少量线程的低延迟（Latency）

GPU 强调：来自许多并发线程的高聚合吞吐量（Throughput）

这两个目标都不是普遍更好的。串行控制密集型代码的 GPU 工作负载可能很差。高度并行的矩阵（Matrix）和张量（Tensor）运算可以成为出色的 GPU 工作负载，因为它们提供许多独立的运算和可以映射到广泛执行资源的常规数据结构。

人工智能工作负载也包含这两种行为。密集矩阵乘法（Matrix Multiplication）暴露了巨大的常规并行性。削减需要协调。嵌入（Embedding）查找可能涉及不规则的内存访问（Memory Access）。自回归推理服务（Serving）在每个解码（Decode）步骤中可以对每个请求进行少量工作。因此，相同的 GPU 对于一个内核（Kernel）来说可能非常高效，而对于下一个内核则没有得到充分利用。

“GPU = 并行，CPU = 串行”太粗糙了。现代 CPU 是并行的，GPU 执行控制流。

重要的区别在于每个架构所支持的并行工作的数量、粒度和调度方式。

3 从内核启动（Kernel Launch）到硬件工作

对应原稿第 7 页

CUDA 内核启动（Kernel Launch）会创建线程块网格。每个块都包含线程，并且块被调度到流多处理器（SMs）上。一旦驻留（Resident），块的线程将被划分为 NVIDIA 硬件上的 32 线程线程束（Warps）。[1]

对于包含 T 线程的块，

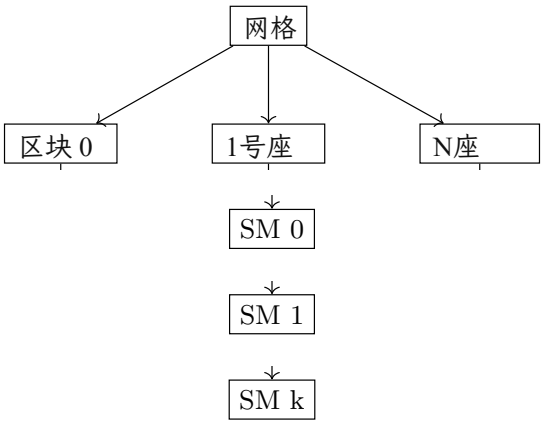
$$\left\lceil \frac{T}{32} \right\rceil .$$

$N_{\text{warps}}=$

32

一个 256 线程块（Thread Block）包含八个线程束（Warps）。一个 300 线程块占用 10 个 warp，最后一个 warp 中只有 12 个有效线程。 CUDA 允许这样的块，但未使用的通道使最终线程束的效率降低。[1]

一个网格包含的块数可能远远多于同时驻留（Resident）的块数。这是正常的。网格描述了逻辑工作。当 SM 资源变得可用时，GPU 接纳块，执行它们，并稍后接纳更多块。一个普通块在其生命周期内被放置在一个 SM 上；如果资源限制允许，多个块可以驻留在一个 SM 上。[1,2]



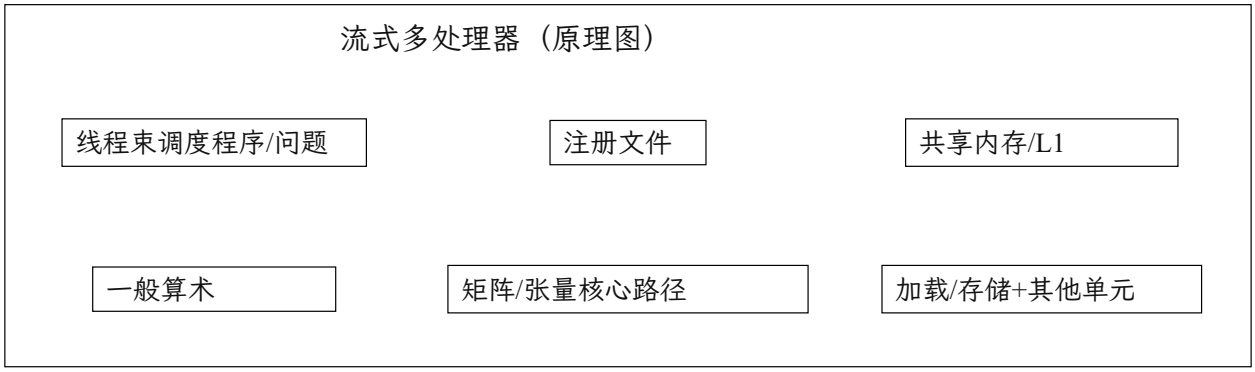
程序员指定网格和块的形状。硬件/运行时决定块在何处运行，但受架构和资源限制。这种分离是 CUDA 的可扩展性思想之一：内核（Kernel）可以描述比当前设备可以同时执行的并行工作多得多的并行工作。[1]

4 流式多处理器（Streaming Multiprocessor）是执行中心

对应原稿第 8 页

流式多处理器（Streaming Multiprocessor）是 NVIDIA 架构中的中央执行引擎。它不仅仅是一组算术单元。它包含保持许多 warp 驻留（Resident）并取得进展所需的资源：寄存器文件（Register File）、共享内存/L1 资源、warp 调度和发布机制、加载/存储路径以及多类功能单元。确切的组织因计算能力而异。[1]

有用的摘要 SM 如下所示：



内核（Kernel）的块消耗有限的 SM 资源。寄存器（Registers）是为线程分配的。共享内存（Shared Memory）是按块分配的。架构限制了常驻块、线程束（Warps）和线程。这些限制共同决定了可以驻留（Resident）的工作量。[1,2]

SM 模型（Model）还解释了为什么“CUDA 核心（CUDA Core）数”是一个不完整的性能（Performance）模型。工作负载不会消耗相同核心的通用池。不同的指令类型使用不同的管道，现代 AI 内核在很大程度上依赖于矩阵（Matrix）指令、加载/存储吞吐量（Throughput）、共享内存行为、同步（Synchronization）和调度。有用的推理单位是整个 SM 加上它的内存提要，而不是单个营销计数。

5 线程束（Warps）和 SIMT

对应原稿第 9 页

在 NVIDIA 线程块（Thread Block）内，线程被组织成 32 个组，称为线程束（Warps）。 CUDA 描述

使用 SIMT 执行：单指令、多线程。每个线程都运行相同的内核（Kernel）程序，但每个线程都有自己的数据，并且可以遵循不同的控制流路径。[1]

SIMT 类似于 SIMD，因为许多通道可以一起执行公共指令，但 CUDA 明确警告这两个概念并不相同。SIMD 公开了具有一个控制流路径的固定宽度向量（Vector）指令。 SIMT 向程序员公开各个线程并将它们分组以供执行。[1]

编程模型（Model）将线程束（Warp）视为在内核（Kernel）中一起进行。硬件实现细节可以更加灵活，特别是在具有独立线程调度的 NVIDIA 架构上，但可移植代码不得依赖于未记录的子线程束行为。 [1]

线程束（Warp）分组有几个后果：

• 块大小为32 的倍数可避免永久填充不足的最终线程束（Warp）；

• 相邻线程经常一起发出内存操作，这就是地址布局影响合并的原因；

• 控制流分歧在线程束（Warp）内是有意义的；

• 许多线程束（Warp）级集体操作需要明确的参与掩码和同步（Synchronization）规则。[1]

线程束（Warp）并不是“32 条指令独立运行”。它是调度/执行分组，32 个 CUDA 线程通常通过它取得进展。线程仍然是逻辑程序实体； warp 是面向硬件的执行组。

6 分流车道和活动车道

对应原稿第 10 页

假设线程束（Warp）执行数据相关分支：

```
if (x > 0) {
  path_A();
} else {
  path_B();
}
```

如果所有活动线程选择同一分支，则线程束（Warp）遵循一条路径。如果一些线程选择 A 而其他线程选择 B，则经纱已发散。 CUDA 的执行模型（Model）必须考虑两条路径，同时仅启用属于当前正在执行的路径的通道。[1]

发散并不意味着内核（Kernel）不正确，并且它不会施加任何固定的惩罚。成本取决于不同路径包含的工作量、它们如何重新聚合以及特定于架构的调度。耐用点更简单：当只有一部分通道为指令执行有用的工作时，warp 的可用通道级吞吐量（Throughput）没有得到充分利用。

分支发散发生在线程束（Warps）内。两个不同的线程束可以采取完全不同的路径，而不会导致它们之间的线程束发散。[1]

Volta 一代 NVIDIA GPU 上引入的独立线程调度使线程调度比旧的锁步实现更加灵活。它不会消除任意分歧，也不会消除使用 CUDA 编程模型（Model）所需的同步（Synchronization）原语的需要。 [1]

分叉也不是车道变得不活跃的唯一原因。最终的部分线程束（Warp）、谓词指令、掩码或边界条件可能会使通道处于非活动状态，而无需传统的 if/else 分割。

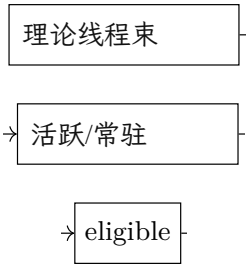
> 安全声明：分歧会减少有用的车道参与。不安全的说法：“每个发散的分支都会自动使线程束（Warp）速度减慢两倍。”实际成本取决于执行的路径和硬件行为。

7 Warp 调度程序和符合条件的 Warp

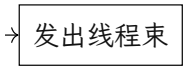
对应原稿第 11 页

仅居住权并不能保证进步。常驻线程束（Warp）可能正在等待内存依赖性、算术结果、同步（Synchronization）或其他事件。下一条指令可以发出的线程束是合格的。调度程序在符合条件的线程束中进行选择并向适当的管道发出指令。 [3]

Nsight Compute 的调度程序模型（Model）非常有用，因为它区分了四种状态：



符合资格



理论驻留（Resident）时间受到启动配置和资源的限制。活动线程束（Warps）是常驻的。合格的线程束是未停滞的活动线程束。发出的线程束是那些实际选择发送指令的线程束。 Nsight 指出，没有合格线程束的周期会导致跳过问题机会，并表明延迟（Latency）隐藏效果不佳。 [3]

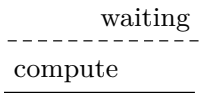
这种区别解释了一个常见的分析意外：内核（Kernel）可以拥有相当可观的占用率（Occupancy），但仍然发出低效，因为在重要时刻符合条件的活动线程束（Warps）太少。相反，如果内核暴露了足够的指令级并行性并保持相关管道的供应，则它可以在适度的占用率下运行良好。

因此，调度程序是架构和性能（Performance）之间的桥梁。它无法制造程序未公开的独立性。它只能在驻留（Resident）且准备好的工作中进行选择。

8 延迟（Latency）隐藏并不是延迟减少

对应原稿第 12 页

GPU 操作有延迟（Latency）。全局内存请求、算术依赖性或同步（Synchronization）可以阻止一个 warp 立即发出其下一条指令。 GPU 通过其他可用的独立工作来应对大部分延迟。[2,3]



计算

等待

加载

使用

线程束（Warp）A

经纱B

计算

经纱C

时间

Warp A 的内存延迟（Latency）并没有消失。机器在部分过程中只是做了 B 和 C 中的有用功

的等待间隔。这就是延迟（Latency）隐藏。[2]

两种独立帮助：

• 线程级并行（TLP）：其他线程束（Warps）有就绪指令；

• 指令级并行性(ILP)：同一个warp 具有独立的操作，可以继续执行而无需等待停止的结果。

这就是为什么入住率很重要，但并不是全部。更多常驻线程束（Warps）为 TLP 创造更多候选者。但是，如果内核（Kernel）已经有足够的准备工作来覆盖重要的延迟，则添加驻留（Resident）可能不会改善运行时间。 [2]

> GPU 通常获胜，不是因为每次内存访问（Memory Access）都是低延迟（Latency）的，而是因为该架构的构建是通过维护和调度许多独立操作来容忍延迟的。

9 驻留、执行和进展

对应原稿第 13 页

经常混淆在一起的三个词应该分开。

常驻意味着块或线程束（Warp）已被接纳到 SM 并且其所需的状态已分配。寄存器（Registers）、共享内存（Shared Memory）、块槽、线程束槽和线程限制限制驻留（Resident）。[1,2]

Active 通常由分析工具用于尚未完成的常驻线程束（Warps）。活跃的线程束可能仍会花费停滞的周期。[3]

合格意味着常驻线程束（Warp）已准备好发出其下一条指令，而不是被依赖性或同步（Synchronization）条件阻止。已发出意味着调度程序实际上选择了该 warp 在该调度机会中调度指令。 [3]

逻辑工作集合 ⊃ 驻留工作集合 ⊃ 可调度工作集合 ⊃ 某一时刻实际发射的工作集合。

一个内核（Kernel）可以启动数百万个逻辑线程，同时只有一个有界集合驻留（Resident）。它可以有许多常驻线程束（Warps），但只有少数符合条件。当一个特定的执行管道饱和而另一个空闲时，它可以具有合格的线程束。

在解释性能（Performance）计数器时，这种分离至关重要。驻留指标无法告诉您哪些功能单元繁忙。基于“任何内核（Kernel）执行”的利用率指标无法告诉您主要管道是否接近峰值。一个子系统的吞吐量（Throughput）指标无法告诉您端到端应用程序时间是否由另一个子系统主导。[3,4]

10 占用：确切的定义

对应原稿第 14 页

入住率是一个居住指标。在 CUDA 中，它是根据 SM 上活动的线程束（Warps）数量相对于 SM 的活动线程束的架构限制来计算的。 [2]

占用率（Occupancy） = 每个 SM 的活动驻留（Resident）线程束（Warps） 每个 SM.的最大驻留线程束

如果架构允许在 SM 上驻留（Resident）最多 64 个线程束（Warps），并且内核（Kernel）配置允许驻留 32 个线程束，则理论占用率（Occupancy）为 50%。

占用率（Occupancy）受到多种有限资源的共同限制：

每个块的线程数和最大常驻线程数；

• 每块线程束（Warps）和最大常驻线程束；

• 最大居民区；

• 常驻线程消耗的寄存器（Registers）；

• 常驻块消耗的共享内存（Shared Memory）；

• 特定于架构的资源分配粒度。[1,2]

一种有限的资源足以阻止其他区块驻留（Resident）。几乎不使用共享内存（Shared Memory）的内核（Kernel）仍然可能受到寄存器（Register）限制。具有适度寄存器使用的内核可能会受到共享内存限制。非常小的块在填充可用线程槽之前可能会达到块计数限制。[1]

因此，占用率（Occupancy）是居住率，而不是做有用工作的算术单元的百分比。 Nsight Compute 甚至提供了专用的占用计算器，因为资源限制和分配粒度的相互作用可能并不明显。 [3]

> 占用率（Occupancy）回答“这次发射可以维持多少曲速驻留？”它没有回答“这个内核（Kernel）与峰值计算吞吐量（Throughput）有多接近？”

11 为什么 100% 入住率不是目标

对应原稿第 15 页

更多的常驻线程束（Warps）可以提高延迟（Latency）容忍度，但性能（Performance）不会随着占用而无限期地提高。一旦内核（Kernel）已经公开了足够的就绪工作来覆盖其重要的停顿，额外的驻留（Resident）可能会增加很少的内容。 [2]

试图迫使入住率上升也可能会产生一种权衡。寄存器（Registers）很有用，因为它们在芯片上保存线程本地状态。减少允许的寄存器计数可能会接纳更多块，但它也会增加指令计数或将值溢出到本地内存。具有足够 ILP 的较低占用率（Occupancy）内核（Kernel）可能会优于溢出或执行更多工作的较高占用率版本。 [2]

占用率越高 $\not\Rightarrow$ 性能（Performance）成比例越高。

低入住率仍然可以作为诊断依据。如果常驻线程束（Warps）太少，则当一个线程束停止时，调度程序的选择就会减少。但正确的问题不是“如何最大限度地提高入住率？”是“现成工作不够充分，阻碍了瓶颈资源的有效利用吗？”

分析应将占用率（Occupancy）与调度程序行为和资源吞吐量（Throughput）联系起来。如果由于没有合格的线程束而经常跳过问题槽，则更多的延迟（Latency）隐藏容量可能会有所帮助。如果相关的计算或内存路径已经接近其可持续极限，那么提高占用率可能没有任何作用。 [3]

> 将占用率（Occupancy）视为约束和诊断，而不是分数。目标是最快的内核（Kernel）；最大入住率只是实现这一目标的一种可能手段。

12 个寄存器（Registers）：快速、线程本地状态

对应原稿第 16 页

寄存器（Registers）是与 SM 关联的片上存储，逻辑上对每个线程都是私有的。编译器将它们用于线程局部值，例如循环变量、地址、累加器和中间算术状态。[1,2]

寄存器（Registers）的重要性有两个相反的原因。首先，它们正是性能（Performance）敏感值通常应该存在的地方：接近执行，而不需要重复传输到更大的内存。其次，寄存器文件（Register File）是有限的，并且在 SM 上的所有常驻线程之间共享。因此，每线程寄存器的高使用量可以减少同时适合的块和线程束（Warps）数量。[1,2]

这就是为什么寄存器（Registers）的使用既是容量问题也是延迟（Latency）问题。内核（Kernel）可以通过在寄存器中保存更多值来提高本地重用，但由于适合的线程上下文较少而失去驻留。最佳点是特定于工作负载的。

寄存器（Registers）也参与依赖链。如果一条指令消耗了前一条指令产生的值，则消费者可能需要等待，直到结果可用。其他独立指令或线程束（Warps）可以隐藏该延迟（Latency）。 NVIDIA 的最佳实践指南专门讨论了通过足够的并行工作隐藏寄存器依赖关系。 [2]

对于 AI 内核，寄存器（Registers）平铺是高性能（Performance）矩阵乘法（Matrix Multiplication）和融合内核的核心，因为部分结果可以在算术管道附近累积。但过度使用寄存器会增加压力。以后的手册将定量地处理登记/占用的权衡；这里的架构教训是，登记册既是高价值资源，也是有限的居住预算。

13 寄存器压力（Register Pressure）、溢出（Spilling）和本地内存

对应原稿第 17 页

当寄存器（Registers）需求太高，或者某些线程本地对象无法保存在寄存器中时，

编译器可以将它们放置在 CUDA 本地内存中。尽管有这个名称，但此存储仅在可见性方面是按线程的；它由设备内存（Device Memory）而不是私有片上内存支持。[2]

寄存器（Register）溢出（Spilling）是值到达本地内存的常见原因之一。 CUDA 还可以在那里放置一些大型或动态索引的线程本地对象。这些访问通过设备内存和缓存路径，因此它们比寄存器访问昂贵得多，并且会增加大量内存流量（Memory Traffic）。 [2]

减少寄存器数量	可能的后果
more blocks may become resident	occupancy may increase
less on-chip thread-local capacity	values may spill
more local-memory traffic	latency / bandwidth cost may rise

更多的块可能会驻留（Resident） 片上线程本地容量更少 本地内存流量（Memory Traffic）更多

占用率（Occupancy）可能会增加 值可能会溢出延迟（Latency）/带宽成本可能会上升

这就是为什么 -maxrregcount 或启动界限是调整控制，而不是自动改进。 NVIDIA 明确警告，即使在增加占用率（Occupancy）的同时，较低的寄存器（Register）限制也会增加溢出（Spilling）。[2]

实际的工作流程是检查编译器资源报告并分析生成的内核（Kernel），而不是假设更少的寄存器（Registers）或更多的占用一定更好。

14 共享内存（Shared Memory）：软件管理的片上存储

对应原稿第 18 页

共享内存（Shared Memory）是显式寻址的片上存储，块内的线程可将其用于协作和数据重用。与硬件管理的缓存不同，软件选择在那里放置什么以及何时在其周围同步。[1,2]

一种常见的模式是平铺：将较大张量（Tensor）的子集从设备内存（Device Memory）加载到共享内存（Shared Memory）中，必要时进行同步，重用该平铺进行多个操作，然后继续。目标不仅仅是使用“更快的内存”。目标是减少从层次结构中较远的位置进行昂贵的重复移动。

共享内存（Shared Memory）还消耗 SM 驻留（Resident）预算。请求大量分配的块可能会阻止其他块同时驻留。因此共享内存可以提高局部性，同时减少占用。这两种效果都是真实存在的。[1,2]

在最新的 NVIDIA 架构上，共享内存（Shared Memory）和 L1 功能可以共享或分区统一的片上数据缓存资源，但确切的大小和组织因计算能力而异。该实现事实不会将共享内存转变为普通缓存：程序员仍然显式分配和寻址共享内存。[1]

同步（Synchronization）是成本模型（Model）的另一部分。如果线程通过共享内存（Shared Memory）进行协作，则可能需要 __syncthreads() 等屏障来确保正确性。因此，必须权衡数据重用优势与存储压力、存储体访问行为和同步开销。

15 Cache：L1和L2不是共享内存（Shared Memory）

对应原稿第 19 页

GPU 还包含硬件管理的缓存。在 CUDA 的设备内存（Device Memory）模型（Model）中，全局内存流量（Memory Traffic）可以通过缓存结构，包括每个 SM L1 功能和设备范围的 L2 缓存。确切的政策和能力取决于架构。[1]

关键区别在于控制：

	共享内存	缓存
management	explicit in program	hardware-managed
scope	block-local; cluster DSM if supported	transparent hierarchy
use	deliberate staging / reuse	temporal / spatial locality
residency	allocation can limit blocks	no per-block shared-memory budget

管理范围 使用居住地

在程序块局部中显式；集群 DSM 如果支持故意分段/重用分配可以限制块

硬件管理的透明层次结构时间/空间局部性无每块共享内存预算

缓存可以减少到达 HBM/设备 DRAM 的流量，但不能保证保留每个有用的工作集（Working Set）。访问模式、重用距离、争用和架构都很重要。

这就是为什么“张量（Tensor）适合 GPU 内存”几乎没有提到内核（Kernel）性能（Performance）。容量告诉我们数据是否可以驻留在内存系统中的某个位置。性能取决于数据移动的频率、芯片上捕获的重用程度以及工作集（Working Set）和访问模式是否与硬件匹配。

以后的手册将对层次结构进行定量处理。对于 AS01，请记住这些角色：寄存器（Registers）是线程本地状态，共享内存（Shared Memory）是显式片上协作/重用，缓存是硬件管理的局部性，设备内存（Device Memory）在距执行管道更远的距离处提供大容量。

16 HBM 和设备内存（Device Memory）：容量不是带宽

对应原稿第 20 页

CUDA 的编程模型（Model）使用全局内存（Global Memory）来存储对所有 SMs 可见的设备附加 DRAM。许多数据中心人工智能加速器都使用 HBM 技术实现这种大容量内存，因此人工智能系统讨论经常使用“HBM”作为 GPU 内存的远大容量级别的简写。[5,10]

两个数量必须分开：

• 容量：可以存储多少字节；

• 带宽：在给定的访问模式和工作负载下每秒可以传输多少字节。

模型（Model）可以完全适合设备内存（Device Memory），但仍然运行缓慢，因为内核（Kernel）通过 HBM 接口反复移动大量数据。相反，内核可能具有较小的内存占用（Memory Footprint），但对延迟（Latency）敏感，因为它的访问是稀疏的或相关的。

峰值内存带宽（Memory Bandwidth）也是一个上限，而不是一个承诺。有效带宽取决于访问合并、缓存行为、指令混合、未完成的请求和其他详细信息。 CUDA 的最佳实践指南建议测量有效带宽，而不是假设已达到理论规范。[2]

> “Fits in HBM”是一个容量声明。“HBM 已饱和”是一个带宽声明。“内核（Kernel）正在等待内存”是一个延迟（Latency）/依赖性声明。它们是相关的但不能互换。

17 内存层次结构（Memory Hierarchy）是一个数据移动问题

对应原稿第 21 页

一个有用的面向人工智能的内存层次结构（Memory Hierarchy）是：



层次结构不仅仅是“快”和“慢”位置的排名。每个级别都有不同的容量、范围、管理策略、带宽和访问语义。性能（Performance）问题是在级别之间移动多少字节以及从附近的数据中提取多少有用的计算。

> FlashAttention 是一个有用的例子：它计算精确的稠密注意力（Dense Attention），但改变执行计划。中间块在计算单元附近被消耗和重用，而不是通过 HBM 重复往返大型中间块。[12]

同样的想法也出现在 GEMM 平铺、融合元素内核、卷积、约简和许多编译器优化中。重用很重要，因为一次移动一个值并多次使用它比从更远的地方重复获取相同的值要便宜。

这也解释了为什么单独的 FLOP 计数是一个很差的运行时间预测器。两种实现可以执行类似的算术，但生成不同的内存流量（Memory Traffic）。工作负载的数据移动（Data Movement）是其执行算法的一部分，即使纸上的数学表达式根本没有提到内存。

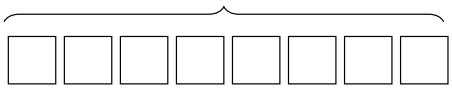
18 合并：线程束（Warp）如何请求内存

对应原稿第 22 页

当 warp 中的线程发出全局内存访问时，GPU 将它们的请求合并到内存中

交易。 CUDA 将此称为合并。高效访问使用每个事务引入的字节，而不是获取包含少量请求数据的许多段。[1,2]

几个连续的段：有效利用传输的字节



分散模式可能需要线程束（Warp）从未使用的更多事务和传输字节。即使程序请求相同数量的有用值，这也会浪费带宽。[2]

因此，合并将线程映射与内存性能（Performance）联系起来。读取相同的张量（Tensor）可能很便宜也可能很昂贵，具体取决于相邻通道遍历的维度以及数据的布局方式。 AS07将对此进行深入处理； AS01 只需要了解架构：内存流量（Memory Traffic）是由线程组共同生成的，因此地址模式很重要。

19 执行管道：指令不使用一种资源

对应原稿第 23 页

SM 包含多个执行路径。浮点（Floating-point）加法、整数运算、加载/存储指令、特殊函数和矩阵（Matrix）乘法累加并非全部由一个无差别的“核心”提供服务。确切的管道计数和映射在 GPU 代之间发生变化。[1,5,6]

这很重要，因为利用率可能是不对称的。一个内核（Kernel）可能会使一个子系统饱和，而其他子系统大部分处于空闲状态：

• 密集的GEMM 可能会花费大量的有用时间来馈送矩阵（Matrix）乘法硬件。

• 内存复制可以接近内存系统限制，同时执行很少的算术；

• 元素内核（Kernel）可能会受到内存流量（Memory Traffic）或指令问题的限制，而不是张量核心（Tensor Cores）；

• 减少可能受到依赖性和同步（Synchronization）的限制。

因此，“GPU 具有未使用的计算”并不是完整的诊断。哪个计算？哪个指令路径？内存已经是瓶颈了吗？是否存在符合条件的线程束？内核（Kernel）是否太小而无法提供足够的设备范围并行性？

Nsight Compute 的吞吐量（Throughput）模型（Model）通过报告相对于峰值的高级计算和内存吞吐量并暴露较低级别的贡献者来反映这一现实。 [3]

> GPU 是耦合子系统的集合。性能（Performance）调优意味着找到限制特定工作负载有用进度的子系统，而不是试图使图中的每个块都读取 100%。

20 个 Tensor 核心：矩阵（Matrix） MMA，而不是神奇的 AI 核心

对应原稿第 24 页

Tensor Core 是用于矩阵（Matrix）乘法累加 (MMA) 运算的专用硬件。 NVIDIA 的编程接口公开了以下形式的概念操作

$$D = A \times B + C.$$

支持的图块形状、操作数格式、累积格式和执行范围取决于 GPU 生成。[7,11]

张量核心（Tensor Cores）对于人工智能来说非常重要，因为训练（Training）和推理（Inference）包含大型 GEMM。但它们并不能加速模型（Model）中的每条指令。索引、控制流、许多归约、元素转换、内存移动、同步（Synchronization）和主机端工作使用其他资源。

“Tensor Core”这个词也隐藏着代际更迭。早期的 CUDA 接口公开了线程束（Warps）级矩阵（Matrix）运算。 Hopper 为某些高性能（Performance）路径引入了异步 warpgroup MMA 指令，其中四个连续的 warp 作为一个 warpgroup 进行协作。数据中心 Blackwell SM100 为部分矩阵管道添加了更新的 tcgen05.mma 指令和张量（Tensor）内存 (TMEM)。[6,11]

这些差异是围绕持久原则的实现细节：当工作负载以支持的矩阵（Matrix）图块和数据格式表示并且周围的数据移动（Data Movement）保持单元供电时，专用矩阵硬件可实现高吞吐量（Throughput）。

> 张量核心（Tensor Cores）提高了支持的矩阵（Matrix）算术的上限。它们不会消除 AI 工作负载中的内存流量（Memory Traffic）、启动开销（Launch Overhead）、同步（Synchronization）或非矩阵工作。

21 Precision改变可用的执行路径

对应原稿第 25 页

峰值算术吞吐量（Throughput）取决于数据类型。对于 FP64、FP32、TF32、BF16、FP16、FP8、整数格式和较新的窄格式，GPU 可以具有非常不同的理论速率。矩阵（Matrix）硬件还支持因代而异的输入和累加（Accumulation）精度（Precision）的特定组合。[5,6,7,10]

这意味着诸如“X TFLOPs”之类的规范是不完整的，除非它至少标识：

• 操作类；

• 输入数据类型；

• 累积模式；

• 稠密与结构化稀疏假设（如果适用）；

• 硬件生成（Generation）。

精度（Precision）不仅仅是一个性能（Performance）旋钮。它会改变数值行为、内存占用（Memory Footprint）、内存流量（Memory Traffic），有时甚至会改变哪些执行指令是合法的。 FP16/BF16 训练（Training）内核（Kernel）可以使用 FP64 科学内核不可用的矩阵（Matrix）路径。 FP8 路径可能需要缩放（Scaling）和错误管理技术，而更高精度的路径则不需要。

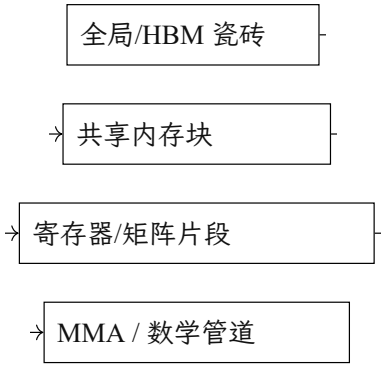
对于系统推理，请将两个声明分开：硬件支持高吞吐量（Throughput）低精度路径，并且该模型（Model）或内核（Kernel）可以安全有效地使用它。首先是架构。第二个取决于软件和数字要求。

22 GEMM 如何映射到 GPU 层次结构

对应原稿第 26 页

矩阵乘法（Matrix Multiplication）是一个有用的架构示例，因为高性能（Performance）实现有意镜像 GPU 层次结构。 CUTLASS 将 GEMM 描述为块的层次结构：设备范围的工作被分解为线程块块，然后是线程束（Warp）/线程束组块，然后是线程或矩阵指令片段。 [11]

简化的数据流是：



平铺的重点是重复利用。从全局内存（Global Memory）加载的图块在被逐出之前可以在许多乘法累加操作中重复使用。寄存器（Registers）保存靠近数学单元的累加器或片段。共享内存（Shared Memory）提供了一个协作的暂存区域。[11]

现代一代添加了更多异步机制，以将数据移动（Data Movement）与矩阵（Matrix）计算重叠。 Hopper 的 WGMMA/TMA 生态系统和数据中心 Blackwell SM100 的 tcgen05/TMEM 路径就是示例。[5,6,11]

数学 GEMM C = AB 没有描述任何这些。实施时间表决定了图块存在的位置、它们何时移动、有多少线程束（Warps）协作以及是否持续供应昂贵的矩阵（Matrix）单元。方程式和执行之间的差距是人工智能系统工程的中心主题之一。

23 指令级和线程级并行

对应原稿第 27 页

GPU 可以使用多种规模的并行性来隐藏延迟（Latency）。

线程级并行性 (TLP) 提供了许多线程束（Warps）。如果 Warp A 停止，调度程序可能会从 Warp 发出

B.

指令级并行性 (ILP) 在一个线程/线程束（Warp）内提供独立操作。如果一个依赖链正在等待，则另一条独立指令可能可用。

这些机制相互作用。高占用率（Occupancy）会增加可贡献 TLP 的线程束（Warps）池。丰富的 ILP 可以减少保持管道繁忙所需的常驻线程束数量。这是低入住率不会自动变慢的原因之一。 NVIDIA 的最佳实践指南明确指出，即使在较低的占用率下，足够的 ILP 有时也可以覆盖延迟（Latency）。[2]

依赖性很重要，因为指令无法在结果准备好之前使用该结果。记忆依赖只是来源之一；算术管道也有延迟（Latency）。安排尝试通过独立工作来填补空白。[2,3]

实际结果是无法仅根据线程数来预测性能（Performance）。如果一万个逻辑线程都达到相同的依赖模式，它们仍然可能会暴露出较差的就绪工作行为。相反，精心流水线化的内核（Kernel）可以使专用资源忙于较小的驻留（Resident）集。

24 同步（Synchronization）会暴露空闲时间

对应原稿第 28 页

线程之间进行协作，因此某些内核需要同步（Synchronization）。 CUDA 的 __syncthreads() 是块范围的屏障：块中每个未退出的线程必须到达相同的屏障实例或退出，然后等待线程才能继续。它还提供参与线程之间所需的内存排序。[1]

屏障对于“加载共享内存块，然后让每个线程使用它”等模式的正确性至关重要。但障碍可能会暴露不平衡。如果一些线程束（Warps）提前到达障碍物而其他线程束仍在工作，则早期线程束会等待。

因此，同步（Synchronization）成本不仅仅是指令本身。它包括当线程等待最慢的所需参与者时变得不可用的有用工作量。

现代 CUDA 还提供线程束（Warp）级同步（Synchronization）、协作组、异步屏障和特定于体系结构的机制，允许更复杂的生产者/消费者管道。 [1]

高性能（Performance）人工智能内核使用这些工具来重叠阶段，但基本规则仍然存在：同步（Synchronization）在工作人员之间创建依赖关系。调度必须尊重这些依赖性，并且不必要的同步会减少可用的并行性。

> 如果大多数常驻 warp 都在同一同步（Synchronization）点等待，则具有许多常驻 warp 的内核（Kernel）仍然可能具有较差的延迟（Latency）隐藏。居住权只有在创造现成工作时才有用。

25 内核（Kernel）启动和异步执行

对应原稿第 29 页

CUDA 内核（Kernel）启动相对于主机线程是异步的：将内核加入队列后，CPU 可以继续，而设备工作仍处于挂起状态。需要结果或想要测量完成的内核时间的代码必须在适当的点同步。[1]

这会影响性能（Performance）和测量。除非正确处理同步（Synchronization），否则放置在启动周围的简单 CPU 计时器可以测量大部分启动开销（Launch Overhead）而不是内核（Kernel）运行时。因此，CUDA 的最佳实践指南将 CPU 时序与 GPU 事件时序区分开来，并强调 API 的异步行为。 [2]

当工作被分割成许多微小的内核（Kernel）时，启动开销（Launch Overhead）变得尤其重要。 GPU 可能具有巨大的算术能力，但如果每次启动执行的工作太少，主机调度和内核边界可能会在实际耗时（Wall-clock Time）内变得可见。 CUDA 图和融合是解决此问题部分内容的后续系列主题。

当资源和依赖性允许时，独立流可以重叠。较新的 GPU 机制还可以将数据移动（Data Movement）与内核内部的计算重叠。[1,5,11]

主机和设备执行解耦；仅在正确性或测量需要时进行同步。

26 “GPU 利用率（GPU Utilization）”的实际含义是什么

对应原稿第 30 页

在 NVML 中，常见的设备 GPU 利用率（GPU Utilization）值是一种活动时间统计数据：在每个样本窗口中，它记录 GPU 至少有一个内核（Kernel）执行的时间份额。 [4]

该指标并不意味着：

• 100% 的算术单元繁忙；

• 张量核心（Tensor Cores）已饱和；

• HBM 带宽已饱和；

• 每个SM 都已被占用；

• 工作负载接近最佳吞吐量（Throughput）。

内核（Kernel）可以连续执行，但仍然会导致 GPU 的大部分内容未得到充分利用。例如，它可能只使用 SMs 的一小部分子集，发出效率低下，或者受到一个管道的限制而其他资源闲置。

NVML 的内存字段同样是基于时间的：它记录发生全局设备内存读取或写入的样本窗口的份额。它不应被解释为实现的带宽除以峰值带宽。[4]

为了优化，Nsight Compute 等内核（Kernel）分析器公开了更具体的资源吞吐量（Throughput）、占用率（Occupancy）、调度程序、缓存和内存指标。 [3]

仪表板可以正确显示 100%“GPU 利用率（GPU Utilization）”，而内核（Kernel）距离对其工作负载至关重要的性能（Performance）上限还很远。

27 占用率（Occupancy）、活动量和吞吐量（Throughput）不同

对应原稿第 31 页

GPU 工具中可以出现几个百分比，它们回答不同的问题。

公制系列	它试图描述什么
occupancy	resident warp fraction relative to the architectural limit
NVML GPU utilization	share of the sample window with kernel execution
active SM / pipeline activity	whether a unit was active over measured cycles
eligible / issued warps	scheduler readiness and actual issue opportunities, separate from residency
吞吐量与峰值	achieved activity relative to an architecture-specific ceiling

占用 NVML GPU 利用率（GPU Utilization） 活动 SM / 符合条件的管道活动 / 发出的线程束（Warps）

相对于架构限制的驻留（Resident）线程束（Warp）分数 与内核（Kernel）执行的示例窗口的份额 单元是否在测量的周期内处于活动状态 调度程序准备情况和实际问题机会，与相对于架构特定上限的驻留实现的活动分开

Nsight Compute 的光速分析将计算和内存吞吐量（Throughput）报告为适当峰值速率的百分比。它的调度程序部分分别报告活动的、合格的和已发出的线程束（Warp）行为。 [3]

这些数字可以不一致，但并不矛盾。内核（Kernel）可能具有 100% NVML 利用率（因为它连续运行）、50% 占用率（Occupancy）（因为寄存器（Register）使用限制驻留时间）、高内存吞吐量（Throughput）（因为 HBM 接近其上限）以及适度的算术吞吐量（因为内核受带宽限制）。

这种组合可以描述一个高效的内核（Kernel）。如果内存已经成为限制资源，那么尝试提高算术利用率将无济于事。

> 只有在您询问指标测量的数量、时间间隔以及相对于哪个分母之后，指标才有意义。

28 峰值 TFLOP 与实现的吞吐量（Throughput）

对应原稿第 32 页

峰值 TFLOP 是指定假设下特定操作和精度（Precision）的硬件上限。它不是应用程序吞吐量（Throughput）。[2,5,6]

为了接近峰值矩阵（Matrix）吞吐量（Throughput），工作负载必须提供支持的数据类型和切片形状、足够的并行工作、高操作数重用、足够的指令发出以及保持矩阵单元供应的数据管道。真实模型（Models）还执行非矩阵内核并产生内存移动、同步（Synchronization）、通信、启动和框架开销。

> 对于相同的操作、精度（Precision）和计数约定，硬件峰值是持续算术吞吐量（Throughput）的上限。端到端应用程序性能（Performance）还反映了内存移动、同步（Synchronization）、通信、内核（Kernel）启动和非矩阵（Matrix）工作。

基准范围很重要。测量一个 GEMM 内核（Kernel）的微基准不能直接与推理服务（Serving）系统的词元（Tokens）进行比较。一种数据类型（Dtype）所达到的 TFLOP/s 数无法与另一种数据类型的峰值相比较。供应商的稀疏峰值与密集峰值不同。

Nsight Compute 的吞吐量（Throughput）指标非常有用，因为它们根据特定于架构的峰值速率显式标准化测量的子系统活动，同时仍然要求工程师确定哪些资源实际上限制了内核（Kernel）。 [3]

> 切勿将峰值数字视为“GPU 将为我的模型（Model）提供此值”。 Peak是特定硬件路径的上限；应用程序吞吐量（Throughput）包括产生结果所需的一切。

29 计算限制和内存限制是工作负载属性

对应原稿第 33 页

内核（Kernel）需要算术和数据移动（Data Movement）。一个有用的一阶量是所选内存级别的算术强度（Arithmetic Intensity）：

跨该内存级别移动的操作字节.

$$I =$$

最初的 Roofline 公式使用片外内存流量（Memory Traffic）（操作强度）将可达到的性能（Performance）与峰值计算和内存带宽（Memory Bandwidth）联系起来。层次结构变体可以将相同的推理风格应用于其他级别。[13]

高强度内核重复使用数据足够多，以至于算术吞吐量（Throughput）可能成为限制上限。低强度内核移动大量字节以进行相对较少的计算，并且更可能受到内存带宽（Memory Bandwidth）的限制。确切的边界取决于硬件。[13]

AS01 中有两个注意事项很重要。

1.“内存限制”并不意味着“使用大量内存容量”。这意味着进度受到测量工作负载下内存系统资源的限制。

2. 内核（Kernel）并不是通过名称永久地受计算限制或内存限制。形状、数据类型（Dtype）、批大小（Batch Size）、缓存行为、融合和 GPU 生成可以改变瓶颈。

完整的 Roofline 模型属于 AS04。对于架构来说，这个教训就足够了：GPU 的计算和内存系统提供了单独的上限。提高非限制上限并不一定会使工作负载更快。

30 个 AI 工作负载对 GPU 的不同部分产生压力

对应原稿第 34 页

“人工智能工作负载”不是一种绩效制度。

大型密集GEMM可以通过平铺和重用来实现高算术强度（Arithmetic Intensity），从而使矩阵（Matrix）吞吐量（Throughput）和周围的数据管道成为中心。

逐元素内核可能只对每个元素执行少量操作，因此花费大量成本来移动数据。融合可以通过避免中间张量的往返来提供帮助。

归约和规范化引入了依赖关系和集体行为，这与令人尴尬的并行元素工作不同。

嵌入（Embedding）或索引工作负载可能具有不规则的访问模式，从而减少局部性或合并。

注意力机制（Attention）包含矩阵（Matrix）乘法、softmax/reduction 行为以及潜在的大中间流量。FlashAttention 证明，在不改变稠密注意力（Dense Attention）数学结果的情况下改变 IO 调度可以显着提高性能（Performance）。 [12]

自回归推理服务（Serving）将形状从长提示预填充（Prefill）更改为一个/几个查询（Query）解码（Decode）步骤，因此同一模型（Model）可以在不同阶段对硬件施加不同的压力。确切的瓶颈取决于批处理（Batching）、上下文长度、内核（Kernel）选择和系统设计。

这种多样性就是模型（Model）级“GPU 利用率（GPU Utilization）”数字对于认真诊断来说过于粗糙的原因。推理服务（Serving）跟踪可能会在具有完全不同瓶颈的内核之间交替。

31 NVIDIA 与 AMD：名称不同，原则不变

对应原稿第 35 页

CUDA 术语不是 GPU 的定义。 AMD 公开了类似的广泛概念，但名称不同，宽度和机制也不同。 HIP 明确警告可移植代码不要采用一种固定的线程束（Warp）大小。 AMD 将硬件分组称为波前（Wavefront）；它的大小因架构而异，而 NVIDIA CUDA 的线程束大小为 32。[8]

概念	NVIDIA 语言	AMD 语言
execution engine	Streaming Multiprocessor (SM)	Compute Unit (CU)
thread group	warp	wavefront / warp in HIP abstraction
programmer-managed on-chip storage	shared memory	LDS (Local Data Share)
matrix acceleration	Tensor Core / MMA families	Matrix Core / MFMA families
programming stack	CUDA	ROCm / HIP

执行引擎 线程组 程序员管理的芯片 存储矩阵（Matrix） 加速编程栈

流式多处理器（Streaming Multiprocessor） (SM) 线程束（Warp）共享内存（Shared Memory）

Tensor Core / MMA 系列 CUDA

HIP 抽象中的计算单元 (CU) 波前（Wavefront）/线程束（Warp） LDS（本地数据共享）

矩阵（Matrix）核心 / MFMA 系列 ROCm / HIP

AMD 当前的性能（Performance）指南描述了相同的高级占用率（Occupancy）权衡：寄存器（Registers）和共享/LDS 资源限制并发块/波，而更高的占用率可以帮助隐藏延迟（Latency）。 [9]

因此，架构启示是可移植的：公开并行工作，在硬件组中调度它，提供每个线程和每个块状态，通过层次结构移动数据，并提供专门的执行单元。名称、确切宽度、资源限制和说明是特定于供应商和代的。

32 代架构改变了机制

对应原稿第 36 页

如果一本关于 GPU 架构的手册将一代人的实现与永恒的理论相混淆，它就会变得过时。持久的原则是线程束（Warps）/波前、驻留、调度、数据移动（Data Movement）、执行管道和资源限制。机制不断发展。

Hopper 引入了 TMA、线程块集群和用于高性能（Performance）内核的 warpgroup 级异步 MMA。[5,11]

数据中心 Blackwell SM100 引入了 tcgen05.mma 张量核心（Tensor Core）指令和这些矩阵（Matrix）管道使用的独特张量（Tensor）内存 (TMEM)。 CUTLASS 将这些视为架构条件功能

而不是通用的 Blackwell 或 CUDA 行为，并明确区分数据中心 SM100 与 GeForce Blackwell SM120。[6,11]

AMD cDNA 代际同样演进了矩阵（Matrix）格式、计算单元结构、封装和存储系统。例如，CDNA 3 扩展了 Matrix Core 数据类型支持，并成为 MI300 加速器系列的基础。 [10]

这种演变就是为什么本手册中标记了特定于体系结构的详细信息以及为什么后续性能（Performance）工作必须与实际部署的 GPU 和软件版本相关联的原因。

> 不要将框图记忆为“GPU”。记住不变量，然后查阅架构文档以了解特定于生成（Generation）的实现。

33 有效执行示例：61,440 个逻辑线程

对应原稿第 37 页

考虑使用 CUDA 启动的内核（Kernel）

240 个块，每个块 256 个线程

网格包含

$$240 \times 256 = 61{,}440$$

逻辑线程。因为 NVIDIA warp 包含 32 个线程，所以每个块包含

$$256/32 = 8$$

线程束（Warps），完整的网格包含

$$240 \times 8 = 1{,}920$$

逻辑工作的线程束（Warps）。这 1,920 个线程束并非同时全部驻留（Resident）。当资源可用时，块将在 SMs 上调度。一个 SM 上可以容纳多少个块取决于设备的架构限制和该内核（Kernel）的资源需求，尤其是寄存器（Registers）和共享内存（Shared Memory）。[1,2]

一旦块驻留（Resident），warp 调度程序就会选择符合条件的 warp 并向适当的执行路径发出指令。某些线程束（Warps）可能会等待内存或依赖项，而其他线程束则继续进行。当块完成时，它们的资源被释放，之后的块可以成为常驻的。

此示例区分了三个尺度：

• 61,440 个线程：逻辑并行工作；

• 1,920 个线程束（Warps）：整个网格的 NVIDIA 执行分组；

• 一次常驻线程束（Warps）：受实际设备和内核（Kernel）资源使用的限制。

算术很简单，但区别是基本的。启动大小描述了要完成的工作，而不是瞬时物理并行性。

34 工作占用示例：资源限制相互作用

对应原稿第 38 页

使用故意假设的 SM，以便算术说明该机制，而无需假装描述特定的 GPU。假设它最多允许 64 个常驻线程束（Warps），具有足够的块槽用于以下配置，并且没有额外的分配粒度限制。它包含 65,536 个 32 位寄存器（Registers）和 128 KiB 共享内存（Shared Memory）。

内核（Kernel）每个块使用 256 个线程，每个线程 64 个寄存器（Registers），每个块使用 32 KiB 共享内存（Shared Memory）。

$256/32 = 8$每块线程束（Warps）

每个块的寄存器（Register）需求是

$256 \times 64 = 16{,}384$寄存器,

所以寄存器（Registers）最多允许

$65{,}536/16{,}384 = 4$块.

共享内存（Shared Memory）也允许

128 KiB/32KiB = 4 个块.

因此，在此简化模型（Model）中驻留（Resident）有四个块，即 $4 \times 8 = 32$ 个线程束（Warps），占用率（Occupancy）为 $32/64 = 50\%$.如果共享内存（Shared Memory）减少到 16 KiB，占用率不会自动上升：寄存器（Registers）仍会将驻留限制为四个块。优化非限制性资源不会改变任何事情。这正是占用分析必须识别主动约束而不是单独调整一个数字的原因。[2,3]

35 种常见误解

对应原稿第 39 页

“一个 GPU 只是数千个微型 CPU。”

不会。线程、调度、内存和吞吐量（Throughput）设计根本不同。[1]

“更多的 CUDA 核心（CUDA Cores）意味着相应更快的 AI。”

不会。AI 内核使用不同的管道、内存资源和精度；核心数量本身并不是一个性能（Performance）模型（Model）。

“100% GPU 利用率（GPU Utilization）意味着 GPU 已饱和。” No. NVML常用显示值是活动随时间变化的衡量标准，而不是饱和（Saturation）度得分。[4]

“Tensor Core 加速了整个模型（Model）。”

不会。它们加速支持的矩阵（Matrix） MMA 操作。其他工作使用其他资源。[7,11]

“如果模型（Model）适合 HBM，那么记忆问题就解决了。”

不会。容量、流量、带宽和延迟（Latency）是不同的约束。

“共享内存（Shared Memory）只是一个缓存。”

不会。它由程序显式管理，即使硬件与 L1 共享物理资源也是如此。[1]

“延迟（Latency）隐藏消除了延迟。”

不。它与等待间隔重叠其他工作。[2]

“人工智能内核是受计算限制的。”

有些是。其他的则根据形状和实现而受到带宽、延迟（Latency）、问题、同步（Synchronization）或启动的限制。

36 实用的理解框架（Mental Model）

对应原稿第 40 页

推理 GPU AI 工作负载的一种紧凑方法是：

1.暴露工作。内核（**Kernel**）定义了块和线程的网格。

2. 放置积木。硬件允许块进入 **SMs**，但受寄存器（**Register**）、共享内存、线程、线程束（**Warp**）和块限制。

3. 组建执行小组。 **NVIDIA** 块变成 **32** 线程线程束（**Warps**）；其他供应商使用自己的分组规则。

4. 保持状态关闭。寄存器（**Registers**）和共享内存（**Shared Memory**）保持有用的工作状态，而缓存则减少了层次结构中更下游的流量。

5. 安排准备情况。 **Warp** 调度程序会选择符合条件的工作，因此一个停止的依赖项不需要停止整个 **SM**。

6. 为正确的管道供电。标量/向量（**Vector**）算术、加载/存储路径、矩阵（**Matrix**）单元和其他资源处理不同的指令。

7. 有意地移动数据。合并、平铺、缓存、重用和融合决定了有多少流量到达设备内存（**Device Memory**）。

8. 测量瓶颈。占用率（**Occupancy**）、利用率、带宽、问题效率和实现的计算是不同的指标。使用能回答当前问题的答案。

工程回报是一个更好的问题。不要问“为什么我的 GPU 不是 100%？”，而是问：

哪些资源或依赖关系限制了此工作负载的有用进展，什么证据（Evidence）表明了这一点？

该问题从单个 CUDA 内核（Kernel）扩展到注意力机制（Attention）、模型（Model）训练（Training）、分布式执行和 LLM 推理服务（Serving）。它是理解人工智能系统其余部分的基础。

参考文献和进一步阅读

对应原稿第 41 页

[1] NVIDIA。 CUDA 编程指南，最新文档，2026 年 9 月访问。https://docs.nvidia.com/cuda/cuda-programming-guide/

原题名与书目信息：[1] NVIDIA. CUDA Programming Guide, current documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] NVIDIA。 CUDA C++ 最佳实践指南，13.4 版文档，2026 年 9 月访问。https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

原题名与书目信息：[2] NVIDIA. CUDA C++ Best Practices Guide, Release 13.4 documentation, accessed September 2026. https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA。 Nsight Compute：分析指南和用户指南，最新文档。 https://docs.nvidia.com/ nsight-compute/

原题名与书目信息：[3] NVIDIA. Nsight Compute: Profiling Guide and User Guide, current documentation. https://docs.nvidia.com/nsight-compute/

[4] NVIDIA。 NVML API 参考：nvmlUtilization_t。 https://docs.nvidia.com/deploy/nvml-api/api/structnvmlUtilization_ _t.html

原题名与书目信息：[4] NVIDIA. NVML API Reference: nvmlUtilization_t. https://docs.nvidia.com/deploy/nvml-api/api/structnvmlUtilization_ _t.html

[5] NVIDIA。料斗调整指南。 https://docs.nvidia.com/cuda/hopper-tuning-guide/

原题名与书目信息：[5] NVIDIA. Hopper Tuning Guide. https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA。 CUTLASS：Blackwell SM100 GEMM / tcgen05 功能。 https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionity.html

原题名与书目信息：[6] NVIDIA. CUTLASS: Blackwell SM100 GEMMs / tcgen05 functionality. https://docs.nvidia.com/cutlass/latest/media/docs/cpp/blackwell_functionality.html

[7] NVIDIA。 CUDA 编程指南矩阵（Matrix）/WMMA API；另请参阅历史 Tensor Core 编程模型（Model）的图灵调优指南。 https://docs.nvidia.com/cuda/cuda-programming-guide/

原题名与书目信息：[7] NVIDIA. CUDA Programming Guide matrix / WMMA APIs; see also the Turing Tuning Guide for the historical Tensor Core programming model. https://docs.nvidia.com/cuda/cuda-programming-guide/

[8] AMD。 HIP C++ 语言扩展、warpSize 和跨通道行为、当前 ROCm 文档。https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html

原题名与书目信息：[8] AMD. HIP C++ Language Extensions, warpSize and cross-lane behavior, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/hip_cpp_language_extensions.html

[9] AMD。 HIP 性能（Performance）指南、占用和资源指南、当前 ROCm 文档。https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/performance_guidelines.html

原题名与书目信息：[9] AMD. HIP Performance Guidelines, occupancy and resource guidance, current ROCm documentation. https://rocm.docs.amd.com/projects/HIP/en/latest/how-to/performance_guidelines.html

[10] AMD。 AMD CDNA 3 架构简介，白皮书。 https://www.amd.com/content/dam/amd/en/ 文档 /instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

原题名与书目信息：[10] AMD. Introducing AMD CDNA 3 Architecture, white paper. https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/white-papers/amd-cdna-3-white-paper.pdf

[11] NVIDIA。 CUTLASS 文档：高效的 GEMM、Warpgroup MMA 和 tcgen05 编程指南。 https://docs.nvidia.com/cutlass/latest/

原题名与书目信息：[11] NVIDIA. CUTLASS Documentation: Efficient GEMM, Warpgroup MMA, and tcgen05 programming guides. https://docs.nvidia.com/cutlass/latest/

[12] Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; Ré, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022.https://arxiv.org/abs/2205.14135

原题名与书目信息：[12] Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; Ré, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. https://arxiv.org/abs/2205.14135

[13] Williams, S.; Waterman, A.; Patterson, D. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures. UCB/EECS-2008-134； CACM 2009。 https://www2.eecs.berkeley.edu/Pubs/ TechRpts/2008/EECS-2008-134.html

原题名与书目信息：[13] Williams, S.; Waterman, A.; Patterson, D. Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures. UCB/EECS-2008-134; CACM 2009. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

参考图：针对哪个问题阅读什么内容

对应原稿第 42 页

问题 最佳起始来源

当前的 CUDA 线程/块/warp 执行模型（Model）是什么？

CUDA 编程指南 [1]

占用、寄存器（Registers）、共享内存（Shared Memory）和延迟（Latency）隐藏有何关系？

CUDA 最佳实践指南 [2]

活动的、合格的和已发布的线程束（Warps）在分析中意味着什么？

Nsight 计算 [3]

常见的 NVIDIA“GPU 利用率（GPU Utilization）”百分比到底衡量什么？

NVML nvmlUtilization_t [4]

哪些机制是 Hopper 特有的？ Hopper 调整指南 [5] 和 CUTLASS [11] 哪些机制区分数据中心 Blackwell SM100 类路径和 SM120?

CUTLASS Blackwell / tcgen05 文档 [6,11]

Tensor Core 矩阵（Matrix）运算实际计算什么？

Turing/CUDA 矩阵（Matrix）文档 [7]

为什么可移植的 GPU 代码不应该假设跨供应商的一种线程束（Warp）宽度？

AMD HIP 语言扩展 [8]

AMD 如何描述占用和延迟（Latency）隐藏?

AMD HIP 性能（Performance）指南 [9]

什么是 AMD AI 加速器架构
例子?

AMD CDNA 3 白皮书 [10]

高性能（Performance） GEMM 如何映射到块、线程束（Warps）、共享内存（Shared Memory）和寄存器（Registers）？

CUTLASS GEMM 文档 [11]

为什么更少的内存流量（Memory Traffic）可以让同样注意力机制（Attention）的数学运算更快?

FlashAttention [12]

计算上限和内存带宽（Memory Bandwidth）如何相互作用？

屋顶线模型（Roofline Model） [13]

> 实施支持和硬件细节的变化速度快于架构原则。对于已部署的系统，请将确切的 GPU 生成、工具包、编译器、框架和分析器版本的文档视为权威。

面向人工智能工程师、开发人员和认真学习者的简短技术手册